SLO-01: SLO and SLI Fundamentals

> **Series:** SLO — Service Level Objectives | **Notebook:** 1 of 6 | **Created:** June 2026 | **Last Updated:** 10/12/2026

Overview

Service Level Objectives turn "is the service healthy?" from a matter of opinion into a measured, agreed number. This notebook defines the three building blocks — **SLI**, **SLO**, **error budget** — explains how Dynatrace models them in the modern SLO app, and helps you choose your first handful of SLOs.

This is the conceptual foundation for the series. SLO-02 builds the SLI queries, SLO-03 covers composition and error budgets in depth, SLO-04 covers alerting, SLO-05 covers provisioning SLOs as code, and SLO-06 covers Site Reliability Guardians — the point-in-time deployment-gating counterpart to the ongoing reliability target defined here.

Table of Contents

1. The Three Building Blocks
2. Why SLOs Matter
3. The Dynatrace SLO Model
4. Anatomy of an SLO
5. Choosing Your First SLOs
6. Cross-Series Pointers

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Grail and the SLO app
IAM permissions (SLO app itself)	`slo:slos:read`, `slo:slos:write`, `slo:objective-templates:read` — the gating permissions for the app; a minimal read/write grant is `ALLOW slo:slos:read, slo:objective-templates:read; ALLOW slo:slos:write;`
Data-source permissions	`storage:buckets:read`, `storage:files:read`, plus whichever of `storage:metrics:read` / `storage:logs:read` / `storage:bizevents:read` / `storage:events:read` / `storage:security.events:read` your SLI queries touch (the SLO app's documented list). `storage:spans:read` is not in that list; add it only if an SLI queries spans
Concepts	Basic DQL (see the SPANS and DBMON series); familiarity with your service topology

> **Sources:** Service-Level Objectives (DT docs), permissions table read 10/12/2026.

1. The Three Building Blocks

Term	What it is	Example
SLI — Service Level Indicator	A measured ratio of *good* events to *total* events, expressed as a percentage	99.61% of requests succeeded
SLO — Service Level Objective	A target for an SLI over a defined window	availability ≥ 99.5% over 30 days
Error budget	The allowed shortfall: `100% − target`	0.5% of requests may fail before the SLO is breached

An **SLI** answers *how are we doing right now*. An **SLO** sets *how good is good enough*. The **error budget** converts the gap into a spendable quantity — once it is gone, the SLO is breached, and that is the signal teams act on.

A note on SLAs: an **SLA** (Service Level Agreement) is a contract with consequences (credits, penalties). SLOs are your internal targets, and you almost always set them *stricter* than any SLA so you find out before the customer does.

2. Why SLOs Matter

- **They translate technical health into business language.** "p95 checkout latency under 1s, 99.9% of the month" is something a product owner can reason about.
- **They give you an error budget.** A budget reframes reliability from "zero errors" (impossible, paralysing) to "errors are fine until the budget runs low" — which then drives concrete decisions: keep shipping, or slow down and harden.
- **They are the highest-signal alert source.** Alerting on *burn rate* against the budget catches both the acute outage and the slow leak, with far less noise than threshold alerts on raw metrics (SLO-04).

3. The Dynatrace SLO Model

Dynatrace has two generations of SLO tooling:

	SLO Classic	**Modern SLO app (use this)**
SLI definition	Metric expressions, fixed indicator types	**DQL-based** — any `good ÷ total` query over metrics, spans, logs, or bizevents
API	Service-level Objectives API classic (`/api/v2/slo`)	**SLO Service Public API**
Backing store	classic config; Settings 2.0 schema `builtin:monitoring.slo`	the SLO service itself, not a Settings schema
As code	`dynatrace_slo_v2` Terraform resource (metric-selector SLIs only)	**`dynatrace_platform_slo`** Terraform resource (provider v1.78.0+) / Monaco (v2.22+)

	SLO Classic	Modern SLO app (use this)
Data scope	service metrics	all Grail data types

> **A correction worth reading if you have codified SLOs already (07/31/2026).** This notebook and SLO-05 previously placed `builtin:monitoring.slo` and `dynatrace_slo_v2` in the modern column. They belong to the classic generation: `dynatrace_slo_v2` authenticates with classic `slo.read` / `slo.write` plus `settings.*` scopes and writes through `builtin:monitoring.slo`, whereas the modern app has its own **SLO Service Public API** and a separate `dynatrace_platform_slo` resource using OAuth `slo:slos:*` scopes. The readiness scan additionally flags both `/api/v2/slo` and `builtin:monitoring.slo` as blocked at upgrade.
>
> **Where "blocked at upgrade" comes from — and what the docs say instead (checked 08/28/2026; quote re-read 09/28/2026).** That phrase is the **tenant-side upgrade readiness scan's** classification, read 07/31/2026. It is not documentation: no Dynatrace page states it, and the published upgrade guidance (Upgrade from Service-Level Objectives Classic (DT docs)) is markedly softer — *"An automated upgrade flow is under consideration; however, given the highly customized nature of SLOs, manual review is expected to yield the best results."* The two are not in conflict — a readiness scan reports what your tenant will not carry across, while the docs describe the migration you are expected to perform by hand — but they carry different urgency, so **run the readiness scan against your own tenant** rather than planning from either this notebook or the docs page alone. If your SLOs are in Terraform as `dynatrace_slo_v2`, they are on the classic path — see SLO-05 §2.

New work should use the **modern SLO app** with DQL-based SLIs — it covers every data type and is the path that is actively developed. Dynatrace's own guidance is to still prefer a metric-based SLI over a DQL one where a suitable pre-aggregated metric exists, since metric queries are faster and cheaper to evaluate; use `makeTimeseries` for event/log-derived SLIs where no such metric exists (SLO-02). If you have SLO Classic definitions, Dynatrace publishes an upgrade path.



4. Anatomy of an SLO

Every modern Dynatrace SLO is four decisions:

1. **The SLI query** — a DQL query that returns the good-versus-total ratio. This is the bulk of the work and is covered in SLO-02.
2. **The target** — the percentage you commit to (e.g. `99.5`). Optionally a **warning** level above the target that surfaces "getting close" before an actual breach.
3. **The evaluation window** — *rolling* (e.g. last 30 days, always moving) or *calendar* (this month). Rolling windows give smoother trend signals; calendar windows align to business reporting. SLO-03 covers the trade-off.
4. **What happens on breach** — burn-rate alerting and routing, covered in SLO-04.

The error budget is not a separate decision — it falls out of the target (`100% - target`). The art is in the first three choices.

5. Choosing Your First SLOs

The most common failure mode is too many SLOs, none trusted. Start narrow:

- **Pick 2–4 business-critical services** — the user journeys that matter (checkout, login, search), not every microservice.

- **1–2 SLIs per service** — availability plus either latency or error rate. Resist the urge to measure everything.
- **Set achievable targets first.** Measure the current SLI for a week, then set the target slightly below observed performance. A target you breach on day one teaches the team to ignore SLOs.
- **Tighten over time.** SLOs are living guidance — review quarterly, ratchet up as the service matures.

> **Anti-pattern:** copying "99.9%" onto every service because it sounds rigorous. A target must reflect what the service can actually deliver and what its users actually need.

6. Cross-Series Pointers

- **SLO-02** — building the SLI queries (availability, latency, error rate, custom).
- **SLO-03** — error budgets, burn rate, composite and weighted SLOs.
- **SLO-04** — burn-rate alerting and routing SLO breaches.
- **SLO-05** — provisioning SLOs as code (`dynatrace_platform_slo` with a DQL SLI, Monaco; classic `dynatrace_slo_v2` for existing SLOs only).
- **SLO-06** — Site Reliability Guardians: point-in-time deployment validation gates that complement the ongoing reliability target defined here.
- **AIOPS-02** — anomaly detection; SLO breaches and anomalies are complementary detection inputs.
- **WFLOW** — routing the problems an SLO breach raises.

> **Sources:** Service-Level Objectives (DT docs), SLOs for all data types (Dynatrace News), Upgrade from Service-Level Objectives Classic (DT docs) — the config-as-code version gates in the table above, verbatim: *"Configuration as Code via Terraform overview support the SLO Service Public API since v1.78.0"* and *"Configuration as Code via Monaco overview supports the SLO Service Public API since v2.22."*, SLO Service Public API (DT docs) — the modern app's own API, whose endpoints are `/slos` and `/objective-templates`; the classic surface is a separate reference titled *Service-level Objectives API classic*, which is what makes `/api/v2/slo` the classic path, dynatrace_platform_slo resource (Dynatrace provider docs) — *"covers configuration for platform*

> *service-level objectives"*, Dynatrace SaaS only, OAuth **View SLOs** (`slo:slos:read`) / **Create and edit SLOs** (`slo:slos:write`). platform_slo.md (Dynatrace GitHub) — the same resource doc at its source; the registry page renders client-side, so it is the repository copy that makes the two quotes above machine-verifiable. **Readiness scan:** the "blocked at upgrade" classification for `/api/v2/slo` and `builtin:monitoring.slo` is from the tenant upgrade readiness scan, read 07/31/2026; re-checked 08/28/2026 against both upgrade pages and the API references, where it is not restated.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.